

Week 3 Reflection
TCSS-498 Directed Reading (Raghavi Sakpal)
Seth Campbell

This week, I decided to take a detour of sorts. I decided to move Week 6 up to this last week, getting re-acquainted with C++ and exploring what the ESP-IDF has under the hood. The reason for this is to allow for a slower development of the code and to tackle some of the ‘first-time’ exposures to using the ESPs, integration with different controllers, wireless protocols, and serial protocols. Some of which I spent a lot of time working out over the week. I did spend some time reading through the week 3 topics (Jacobian Inverse, pseudoinverse) as well; however, not nearly as much as I did exploring the coding framework.

The ESP-IDF. As I mentioned last week, two main reasons I chose to use this framework is because it’s oriented more towards professional development and less towards hobby development, but also to pull away from some of the pre-developed libraries oriented towards these specific projects. A few major hurdles occurred while setting up the framework, and to understand how the project structure is set up, specifically how to set up the file tree and dress up the config files so that the project manages all ESPs and will flash them accordingly. Unfortunately, I didn’t understand this was a function of the IDF until I had dug into writing a few of the class files, and it occurred to me that I wasn’t entirely sure how I was going to manage flashing the ESPs themselves.

The ESP chipset utilizes a wireless protocol, ESP-Now. This protocol is simple, unencrypted, and handled directly through function calls. The hardest part of implementing it is understanding how to instantiate it, which the documentation doesn’t tell you that the WIFI must be instantiated, set up, and configured before ESP-Now. Outside of that, the hardest part of the setup was plugging in 3 ESP’s and getting their MAC addresses so I could hardcode them in. Of course, because the other classes require leveraging a class that handles the ESP-NOW communication the must naturally call on it. This was the first reminder that I needed to brush up on my C++ because I had completely forgotten that header (.hpp) files exist. I still need to develop these header files for most of the other classes.

Two major issues I still need to tackle, the first will just take some time to figure out, and that is the serial communication between the ‘primary’ ESP and the computer. The library that handles this is having trouble being installed, and I need to figure out what is preventing this. The second is the issue of communication FROM the servo board and their respective ESP. The servo controllers (internal to each servo) handles developing data about the servo and sending it as a TTL protocol (which is half duplex), the Servo adapter handles not just the power delivery but also managing the translation between the TTL protocol’s half duplex and the ESP’s duplex communication (UART using Rx/Tx pins). Luckily, these servo controllers also handle PWM control. The only communication needed TO the servos is the position requested. So the entire pipeline looks more like:

Function	Board	Signal Type
Servo Position	Servo Controllers	TTL(half duplex)
Translation from TTL to UART	Servo Adapter Board	UART(duplex)
Pull data from TLL protocol	ESP(1)	N/A process signal data
Package and send data to other ESP	ESP(1)	ESP-Now
Receive data from ESP	ESP(2)	N/A process signal data
Send new servo position request	ESP	UART(duplex)
Translate from UART to TTL	Servo Adapter Board	TTL(half duplex)
Translate new position data and adjust PWM	Servo Controllers	TTL(half duplex)

Week 3 Reflection
TCSS-498 Directed Reading (Raghavi Sakpal)
Seth Campbell